



Tribhuvan University

Faculty of Humanities and Social Sciences

Kirtipur, Kathmandu

A Lab Report

On

Discrete Structure

Submitted by

Name: Bishal Rijal

Roll No: 6 (Six)

Tu Register no: 6-2-1133-50-2019

Submitted to

Narayan Dhamala

Patan Multiple Campus

Patandhoka, Lalitpur

Department of MCA

Affiliated to Tribhuvan University

Sign of Internal

.....

Sign of External

.....

September- 2026

Contents

1. WAP in python to display truth table of negation.	1
2. WAP in python to display truth table of conjunction.	1
3. WAP in python to display truth table of disjunction.	2
4. WAP in python to display truth table of implication.	2
5. WAP in python to display truth table of bi-implication.	3
6. WAP in python to verify tautology or not.	3
7. WAP in python to find the GCD between two input numbers using Euclidean algorithm.	4
8. WAP in python to find the GCD between two input numbers using Extended Euclidean algorithm.	5
9. WAP in python to solve the given simultaneous congruence using Chinese Remainder theorem.	6
10. WAP in python to test relation properties: Reflexive, Symmetric, Anti-symmetric and Transitive for a given set and Relation.	7
11. WAP in python to represent relation in matrix form for a given set and relation.	8
12. WAP in python to represent relation in digraph form for a given set and relation.	9
13. WAP in python to determine whether the given relation is equivalence relation or not.	10
14. WAP in python to determine whether the given relation is partial order relation or not.	11
15. WAP in python to find the factorial of an input number using recursion.	12
16. WAP in python to find the nth term of Fibonacci sequence using recursion.	12
17. WAP in python to validate inductive proof	13
18. WAP in python to represent adjacency list and visualize using NetworkX.	14
19. WAP In python to represent adjacency matrix and visualize using NetworkX.	15
20. WAP in python to perform tree traversal (pre-order, in-order and post-order).	16
21. WAP in python to find MST using Kruskal's algorithm.	17
22. WAP in python to construct finite automata that accepts strings only ab and abb.	19

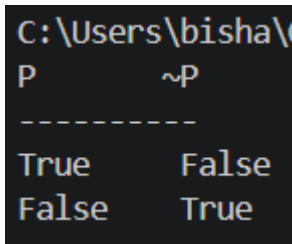
1. WAP in python to display truth table of negation.

```
print("P\t~P")

print("-" * 10)

for P in [True, False]:

    print(P, "\t", not P)
```



```
C:\Users\bisha\OneDrive\Documents>python negation.py
P      ~P
-----
True   False
False  True
```

2. WAP in python to display truth table of conjunction.

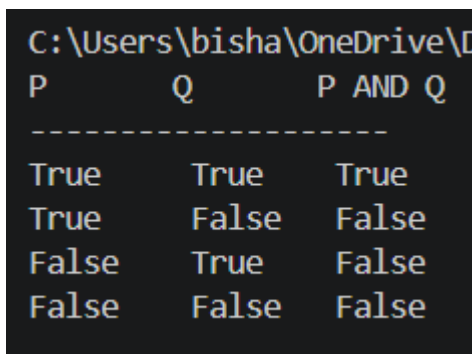
```
print("P\tQ\tP AND Q")

print("-" * 20)

for P in [True, False]:

    for Q in [True, False]:

        print(P, "\t", Q, "\t", P and Q)
```



```
C:\Users\bisha\OneDrive\Documents>python conjunction.py
P      Q      P AND Q
-----
True   True   True
True   False  False
False  True   False
False  False  False
```

3. WAP in python to display truth table of disjunction.

```
print("P\tQ\tP OR Q")
```

```
print("-" * 20)
```

```
for P in [True, False]:
```

```
    for Q in [True, False]:
```

```
        print(P, "\t", Q, "\t", P or Q)
```

```
C:\Users\bisha\OneDrive\
P      Q      P OR Q
-----
True   True   True
True   False  True
False  True   True
```

4. WAP in python to display truth table of implication.

```
print("P\tQ\tP -> Q")
```

```
print("-" * 20)
```

```
for P in [True, False]:
```

```
    for Q in [True, False]:
```

```
        result = (not P) or Q
```

```
        print(P, "\t", Q, "\t", result)
```

```
C:\Users\bisha\OneDrive\
P      Q      P -> Q
-----
True   True   True
True   False  False
False  True   True
False  False  True
```

5. WAP in python to display truth table of bi-implication.

```
print("P\tQ\tP <=> Q")

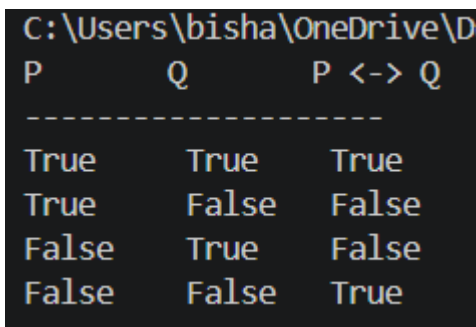
print("-" * 20)

for P in [True, False]:

    for Q in [True, False]:

        result = P == Q

        print(P, "\t", Q, "\t", result)
```



P	Q	P <=> Q
True	True	True
True	False	False
False	True	False
False	False	True

6. WAP in python to verify tautology or not.

```
# Example: P OR NOT P

is_tautology = True

for P in [True, False]:

    result = P or not P

    print("P =", P, "Result =", result)

    if not result:

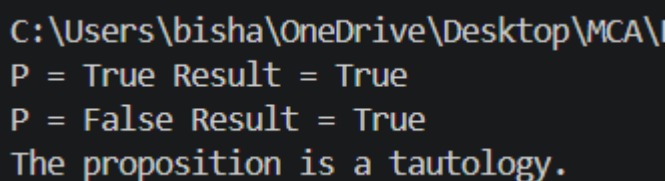
        is_tautology = False

if is_tautology:

    print("The proposition is a tautology.")

else:

    print("The proposition is not a tautology.")
```



```
C:\Users\bisha\OneDrive\Desktop\MCA\
P = True Result = True
P = False Result = True
The proposition is a tautology.
```

7. WAP in python to find the GCD between two input numbers using Euclidean algorithm.

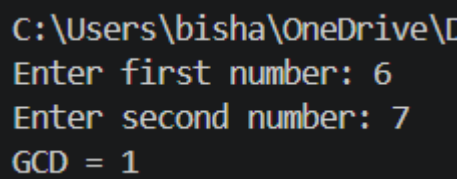
```
a = int(input("Enter first number: "))
```

```
b = int(input("Enter second number: "))
```

```
while b != 0:
```

```
    a, b = b, a % b
```

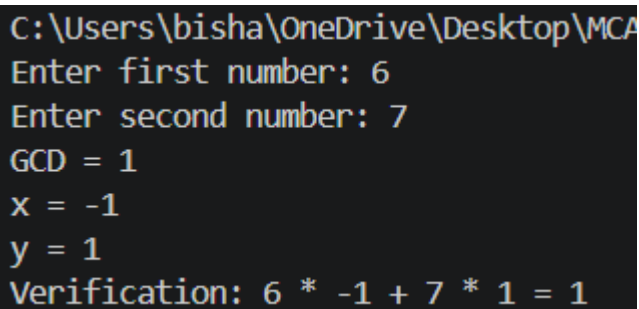
```
print("GCD =", abs(a))
```



```
C:\Users\bisha\OneDrive\...  
Enter first number: 6  
Enter second number: 7  
GCD = 1
```

8. WAP in python to find the GCD between two input numbers using Extended Euclidean algorithm.

```
def extended_gcd(a, b):  
    if b == 0:  
        return a, 1, 0  
    gcd, x1, y1 = extended_gcd(b, a % b)  
    x = y1  
    y = x1 - (a // b) * y1  
    return gcd, x, y  
  
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))  
  
gcd, x, y = extended_gcd(a, b)  
  
print("GCD =", gcd)  
  
print("x =", x)  
  
print("y =", y)  
  
print("Verification:", a, "*", x, "+", b, "*", y, "=", gcd)
```



```
C:\Users\bisha\OneDrive\Desktop\MCA  
Enter first number: 6  
Enter second number: 7  
GCD = 1  
x = -1  
y = 1  
Verification: 6 * -1 + 7 * 1 = 1
```

9. WAP in python to solve the given simultaneous congruence using Chinese Remainder theorem.

```
def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0

    gcd, x1, y1 = extended_gcd(b, a % b)

    x = y1
    y = x1 - (a // b) * y1

    return gcd, x, y

def mod_inverse(a, m):
    gcd, x, y = extended_gcd(a, m)

    if gcd != 1:
        return None

    return x % m

n = int(input("Enter number of congruences: "))

a = []
m = []

for i in range(n):
    a.append(int(input(f"Enter remainder a[{i+1}]: ")))
    m.append(int(input(f"Enter modulus m[{i+1}]: ")))

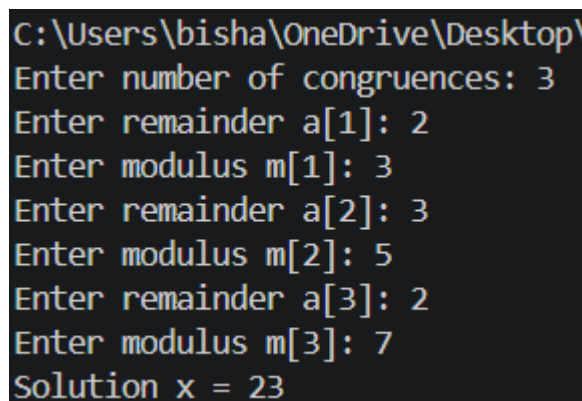
M = 1
for value in m:
    M *= value

x = 0

for i in range(n):
    Mi = M // m[i]
    yi = mod_inverse(Mi, m[i])
    x += a[i] * Mi * yi

x = x % M

print("Solution x =", x)
print("Modulo =", M)
```



```
C:\Users\bisha\OneDrive\Desktop\
Enter number of congruences: 3
Enter remainder a[1]: 2
Enter modulus m[1]: 3
Enter remainder a[2]: 3
Enter modulus m[2]: 5
Enter remainder a[3]: 2
Enter modulus m[3]: 7
Solution x = 23
```

10. WAP in python to test relation properties: Reflexive, Symmetric, Anti-symmetric and Transitive for a given set and Relation.

$A = \{1, 2, 3\}$

$R = \{(1, 1), (2, 2), (3, 3), (1, 2), (2, 1)\}$

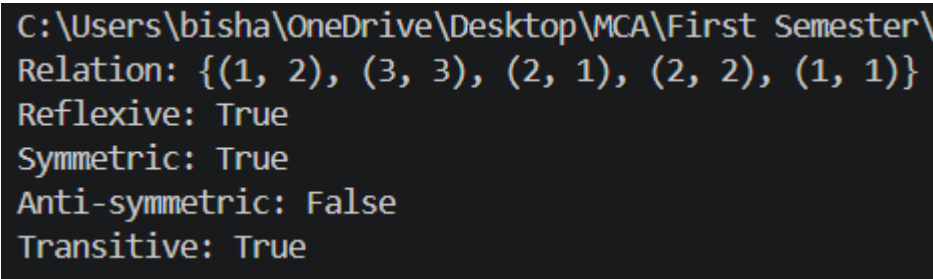
```
reflexive = all((a, a) in R for a in A)
```

```
symmetric = all(
    (b, a) in R
    for (a, b) in R
)
```

```
antisymmetric = all(
    a == b or (b, a) not in R
    for (a, b) in R
)
```

```
transitive = all(
    (a, c) in R
    for (a, b) in R
    for (c, d) in R
    if b == c
)
```

```
print("Relation:", R)
print("Reflexive:", reflexive)
print("Symmetric:", symmetric)
print("Anti-symmetric:", antisymmetric)
print("Transitive:", transitive)
```



```
C:\Users\bisha\OneDrive\Desktop\MCA\First Semester\
Relation: {(1, 2), (3, 3), (2, 1), (2, 2), (1, 1)}
Reflexive: True
Symmetric: True
Anti-symmetric: False
Transitive: True
```

11. WAP in python to represent relation in matrix form for a given set and relation.

```
A = [1, 2, 3]
```

```
R = {(1, 1), (1, 2), (2, 2), (3, 3)}
```

```
print("Relation Matrix:")
```

```
for i in A:
```

```
    for j in A:
```

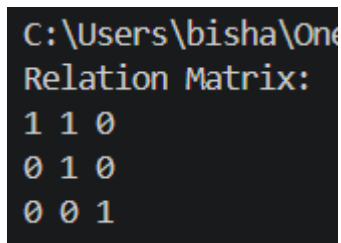
```
        if (i, j) in R:
```

```
            print(1, end=" ")
```

```
        else:
```

```
            print(0, end=" ")
```

```
    print()
```



```
C:\Users\bisha\One
Relation Matrix:
1 1 0
0 1 0
0 0 1
```

12. WAP in python to represent relation in digraph form for a given set and relation.

```
import networkx as nx
import matplotlib.pyplot as plt
```

```
A = [1, 2, 3]
```

```
R = {(1, 1), (1, 2), (2, 3), (3, 1)}
```

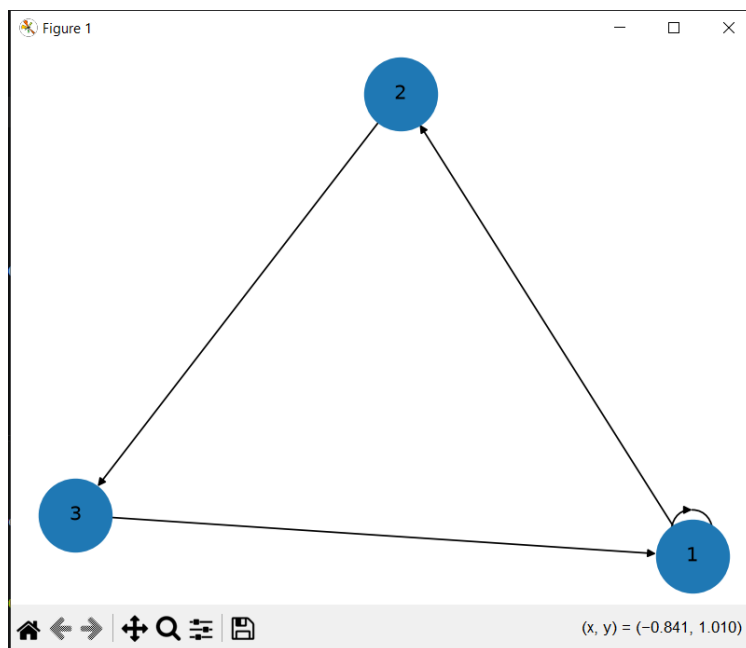
```
G = nx.DiGraph()
```

```
G.add_nodes_from(A)
```

```
G.add_edges_from(R)
```

```
nx.draw(
    G,
    with_labels=True,
    node_size=2000,
    arrows=True
)
```

```
plt.title("Digraph of Relation")
plt.show()
```



13. WAP in python to determine whether the given relation is equivalence relation or not.

$A = \{1, 2, 3\}$

$R = \{(1, 1), (2, 2), (3, 3),$
 $(1, 2), (2, 1),$
 $(2, 3), (3, 2),$
 $(1, 3), (3, 1)\}$

```
reflexive = all((a, a) in R for a in A)
```

```
symmetric = all(  
    (b, a) in R  
    for (a, b) in R  
)
```

```
transitive = all(  
    (a, c) in R  
    for (a, b) in R  
    for (c, d) in R  
    if b == c  
)
```

```
if reflexive and symmetric and transitive:  
    print("The relation is an Equivalence Relation.")  
else:  
    print("The relation is NOT an Equivalence Relation.")
```

```
C:\Users\bisha\OneDrive\Desktop\MCA\First  
The relation is an Equivalence Relation.
```

14. WAP in python to determine whether the given relation is partial order relation or not.

$A = \{1, 2, 3\}$

$R = \{(1, 1), (2, 2), (3, 3),$
 $(1, 2), (1, 3), (2, 3)\}$

```
reflexive = all((a, a) in R for a in A)
```

```
antisymmetric = all(  
    a == b or (b, a) not in R  
    for (a, b) in R  
)
```

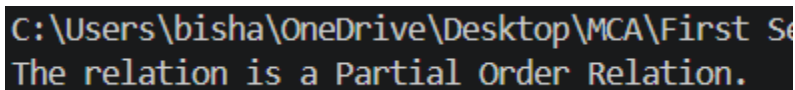
```
transitive = all(  
    (a, c) in R  
    for (a, b) in R  
    for (c, d) in R  
    if b == c  
)
```

```
if reflexive and antisymmetric and transitive:
```

```
    print("The relation is a Partial Order Relation.")
```

```
else:
```

```
    print("The relation is NOT a Partial Order Relation.")
```

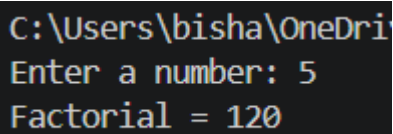


```
C:\Users\bisha\OneDrive\Desktop\MCA\First Se  
The relation is a Partial Order Relation.
```

15. WAP in python to find the factorial of an input number using recursion.

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    return n * factorial(n - 1)
```

```
n = int(input("Enter a number: "))  
  
print("Factorial =", factorial(n))
```

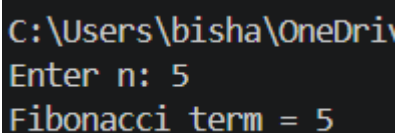


```
C:\Users\bisha\OneDrive  
Enter a number: 5  
Factorial = 120
```

16. WAP in python to find the nth term of Fibonacci sequence using recursion.

```
def fibonacci(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)
```

```
n = int(input("Enter n: "))  
  
print("Fibonacci term =", fibonacci(n))
```

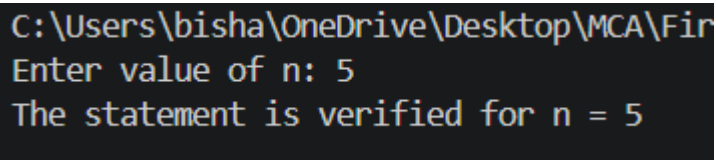


```
C:\Users\bisha\OneDrive  
Enter n: 5  
Fibonacci term = 5
```

17. WAP in python to validate inductive proof

Example: $1 + 2 + \dots + n = n(n+1)/2$

```
def validate_induction(n):  
    lhs = sum(range(1, n + 1))  
    rhs = n * (n + 1) // 2  
  
    return lhs == rhs  
  
n = int(input("Enter value of n: "))  
  
if validate_induction(n):  
    print("The statement is verified for n =", n)  
else:  
    print("The statement is not verified.")
```



```
C:\Users\bisha\OneDrive\Desktop\MCA\Fir  
Enter value of n: 5  
The statement is verified for n = 5
```

18. WAP in python to represent adjacency list and visualize using NetworkX.

```
import networkx as nx
import matplotlib.pyplot as plt

graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C']
}

print("Adjacency List:")

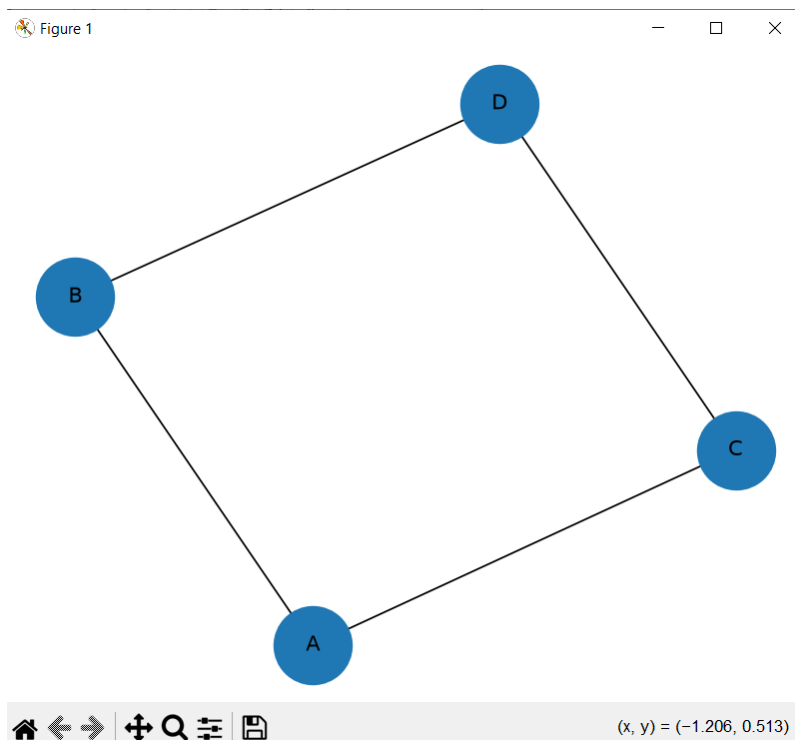
for vertex in graph:
    print(vertex, ":", graph[vertex])

G = nx.Graph()

for vertex in graph:
    for neighbor in graph[vertex]:
        G.add_edge(vertex, neighbor)

nx.draw(
    G,
    with_labels=True,
    node_size=2000
)

plt.title("Graph using Adjacency List")
plt.show()
```



19. WAP In python to represent adjacency matrix and visualize using NetworkX.

```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np

vertices = ['A', 'B', 'C', 'D']

matrix = np.array([
    [0, 1, 1, 0],
    [1, 0, 0, 1],
    [1, 0, 0, 1],
    [0, 1, 1, 0]
])

print("Adjacency Matrix:")
print(matrix)

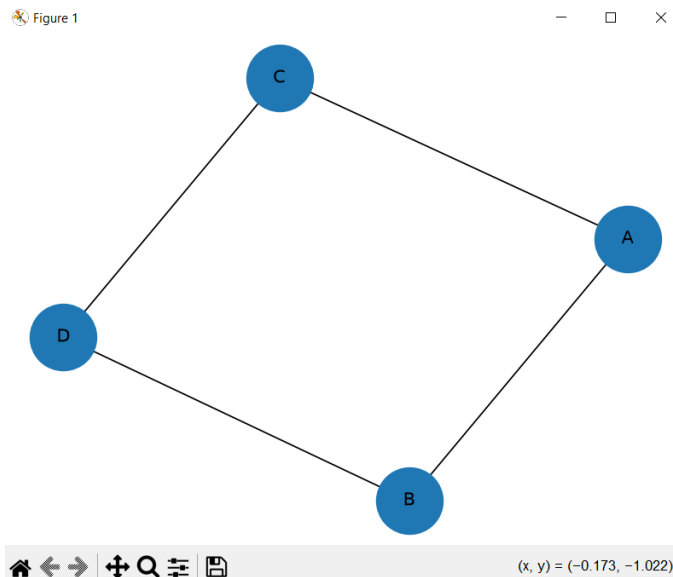
G = nx.from_numpy_array(matrix)

mapping = {
    0: 'A',
    1: 'B',
    2: 'C',
    3: 'D'
}

G = nx.relabel_nodes(G, mapping)

nx.draw(
    G,
    with_labels=True,
    node_size=2000
)

plt.title("Graph using Adjacency Matrix")
plt.show()
```



20. WAP in python to perform tree traversal (pre-order, in-order and post-order).

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)

def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)

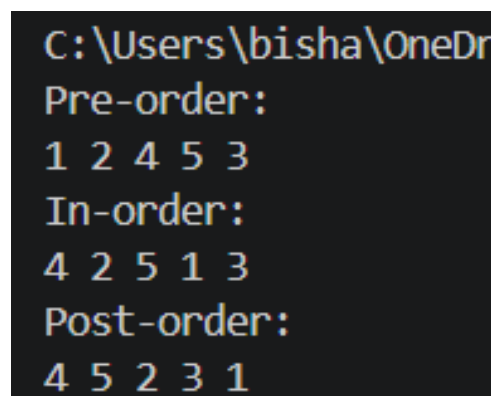
def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")
```

```
root = Node(1)
root.left = Node(2)
root.right = Node(3)
root.left.left = Node(4)
root.left.right = Node(5)
```

```
print("Pre-order:")
preorder(root)
```

```
print("\nIn-order:")
inorder(root)
```

```
print("\nPost-order:")
postorder(root)
```



```
C:\Users\bisha\OneDr
Pre-order:
1 2 4 5 3
In-order:
4 2 5 1 3
Post-order:
4 5 2 3 1
```

21. WAP in python to find MST using Kruskal's algorithm.

```
class Graph:
    def __init__(self, vertices):
        self.V = vertices
        self.edges = []

    def add_edge(self, u, v, w):
        self.edges.append((w, u, v))

    def find(self, parent, i):
        if parent[i] != i:
            parent[i] = self.find(parent, parent[i])
        return parent[i]

    def union(self, parent, rank, x, y):
        xroot = self.find(parent, x)
        yroot = self.find(parent, y)

        if rank[xroot] < rank[yroot]:
            parent[xroot] = yroot
        elif rank[xroot] > rank[yroot]:
            parent[yroot] = xroot
        else:
            parent[yroot] = xroot
            rank[xroot] += 1

    def kruskal(self):
        self.edges.sort()

        parent = list(range(self.V))
        rank = [0] * self.V

        mst = []
        total_weight = 0

        for weight, u, v in self.edges:
            x = self.find(parent, u)
            y = self.find(parent, v)

            if x != y:
                mst.append((u, v, weight))
                total_weight += weight
                self.union(parent, rank, x, y)

        print("Edges in MST:")

        for u, v, weight in mst:
            print(u, "-", v, ":", weight)

        print("Total weight =", total_weight)
```

```
g = Graph(4)
```

```
g.add_edge(0, 1, 10)
```

```
g.add_edge(0, 2, 6)
```

```
g.add_edge(0, 3, 5)
```

```
g.add_edge(1, 3, 15)
```

```
g.add_edge(2, 3, 4)
```

```
g.kruskal()
```

```
C:\Users\bisha\OneDri
```

```
Edges in MST:
```

```
2 - 3 : 4
```

```
0 - 3 : 5
```

```
0 - 1 : 10
```

```
Total weight = 19
```

22. WAP in python to construct finite automata that accepts strings only ab and abb.

```
def finite_automata(string):
    state = "q0"

    for symbol in string:
        if state == "q0":
            if symbol == "a":
                state = "q1"
            else:
                state = "dead"

        elif state == "q1":
            if symbol == "b":
                state = "q2"
            else:
                state = "dead"

        elif state == "q2":
            if symbol == "b":
                state = "q3"
            else:
                state = "dead"

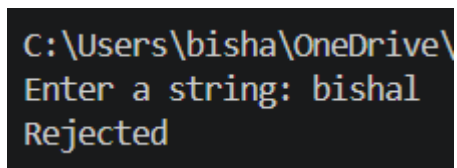
        elif state == "q3":
            state = "dead"

        else:
            state = "dead"

    if state == "q2" or state == "q3":
        return True
    else:
        return False
```

```
string = input("Enter a string: ")
```

```
if finite_automata(string):
    print("Accepted")
else:
    print("Rejected")
```



```
C:\Users\bisha\OneDrive\
Enter a string: bishal
Rejected
```